PROJET CDA · REMPLACEMENT DE STAGE

Rassemble

Dossier technique et preuves

Nicolas Patigny · ENI

Angular · Spring Boot · MongoDB

Livraison technique du 21 octobre 2026. Contexte personnel, validation pédagogique et analyse personnelle à compléter honnêtement avant remise.

Sommaire

ANALYSIS
DESIGN
WIREFRAMES
PROJECT
API
SECURITY
TESTS
EVIDENCE
DEPLOYMENT
ASSESSMENT

Analyse et expression du besoin

Contexte réel connu

Nicolas Patigny prépare une formation CDA à l'ENI. Ce projet remplace son stage et vise à produire une application full-stack démontrable de gestion d'événements sur un périmètre prévu de deux mois. Aucun nom d'entreprise d'accueil, activité commerciale, effectif, organigramme ou tuteur n'a été communiqué. Le dossier présente donc un contexte de projet pédagogique ; ces informations ne doivent pas être inventées.

Besoin

Centraliser la découverte d'événements, leur publication et les inscriptions, avec des droits adaptés aux participants, organisateurs et administrateurs. Éviter une double inscription, une surcapacité ou la modification des événements d'un tiers.

Acteurs

- Visiteur : consulter l'agenda, rechercher, lire un événement, créer un compte.
- Participant (`USER`) : gérer son profil, s'inscrire, se désinscrire avant le début, consulter ses événements à venir et passés, exporter/supprimer son compte.
- Organisateur : publier, modifier et annuler ses événements ; voir les noms des inscrits.
- Administrateur : gérer les comptes/rôles, catégories, tous les événements et les journaux.

L'organisateur ne s'inscrit pas en tant que participant. L'administrateur attribue les rôles ; le formulaire public ne permet jamais de choisir un rôle privilégié.

Décision de stockage

Le PDF prévoyait SQL Server pour le métier et MongoDB pour les journaux. Le porteur a demandé explicitement **MongoDB seul**. Collections : users, events (inscriptions embarquées), categories, audit. Les relations conceptuelles restent documentées dans le MCD mais ne constituent pas un schéma SQL livré. Faire valider par l'enseignant la couverture de la compétence relationnelle si elle est obligatoire pour la certification. Le projet seul ne prouve pas une compétence SQL exécutée.

Périmètre livré

Authentification JWT et révocation, profil, recherche, événements, inscriptions, administration, tableaux de bord, journaux techniques, export et effacement, interface Angular française, tests HTTP/MongoDB et navigateur, livraison Docker, documentation.

Règles et acceptation

ID	Règle	Résultat attendu
RM01	Date future à la création	400 pour une date passée
RM02	Capacité maximale	Une écriture atomique refuse tout dépassement
RM03	Unicité événement/participant	Une seule inscription, même simultanée

ID	Règle	Résultat attendu
RM04	Propriété	403 pour l’organisateur tiers
RM05	Complet	409 pour toute place supplémentaire
RM06	Annulé	Aucune nouvelle inscription
RM07	Administration	401 sans connexion, 403 sans rôle
RM08	Validation serveur	400 avec erreurs de champs
RM09	Modification concurrente	Révision obsolète : 409
RM10	Réduction de capacité	Refus si inférieure aux inscrits
RM11	Compte désactivé/supprimé	Jeton existant inutilisable

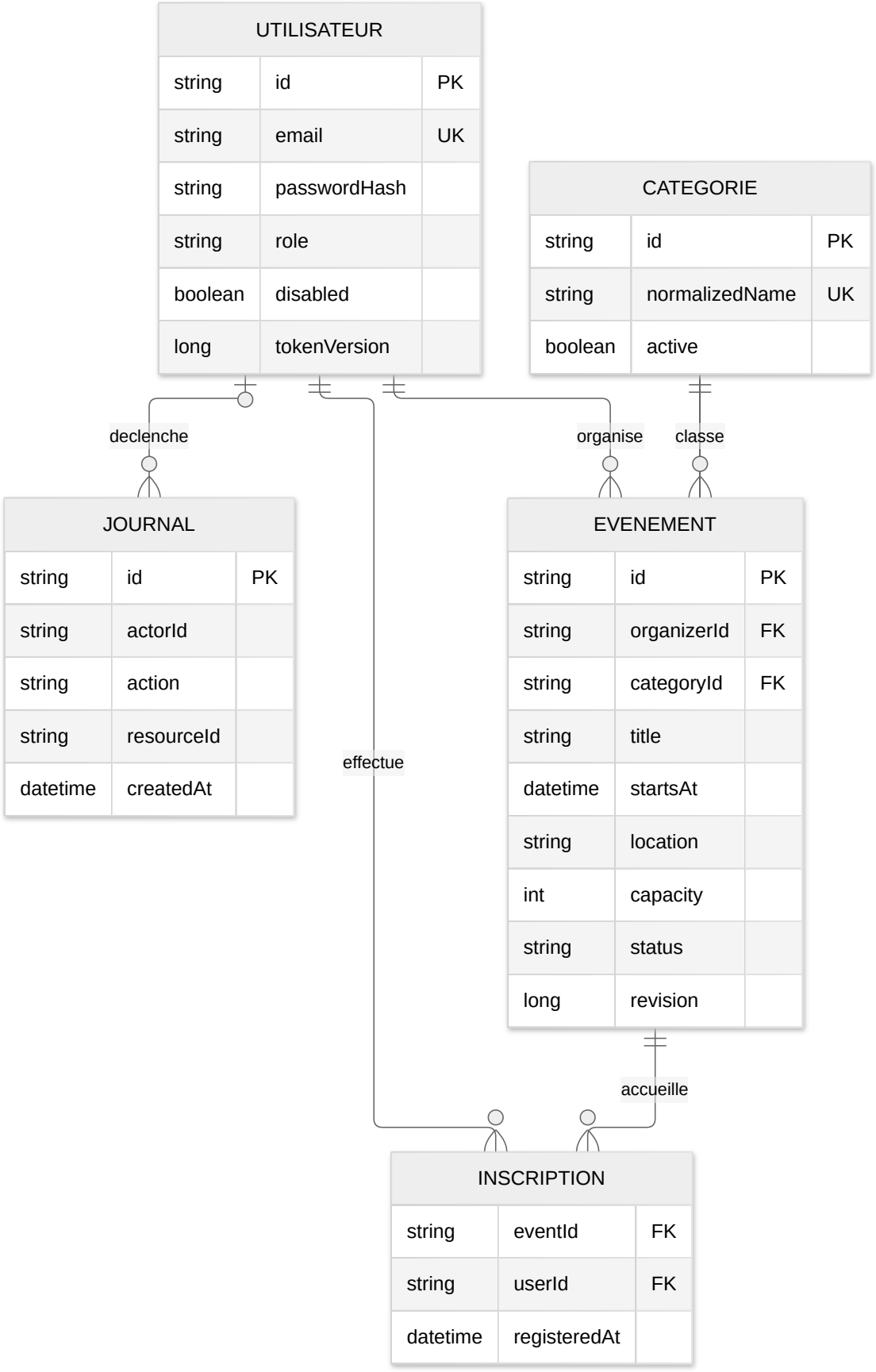
L’annulation d’un événement conserve sa fiche et ses inscriptions afin d’informer les participants. La désinscription retire la réservation ; l’historique affiche les événements passés auxquels l’utilisateur est resté inscrit, pas une archive de chaque réservation annulée. Les journaux conservent temporairement les actions de désinscription.

Limites assumées

Pas de paiement, messagerie, application mobile native, recommandations, e-mails, billets PDF, QR codes, mot de passe oublié ou notifications temps réel. Les places sont bornées à 1 000 pour conserver des documents MongoDB raisonnables. Le MVP n’a pas fait l’objet d’un audit externe ni d’un test de charge de production. Les données de test sont fictives. Les lieux sont des champs structurés au niveau de l’événement, sans référentiel indépendant à administrer.

Conception

Modèle conceptuel de données (indépendant du stockage)



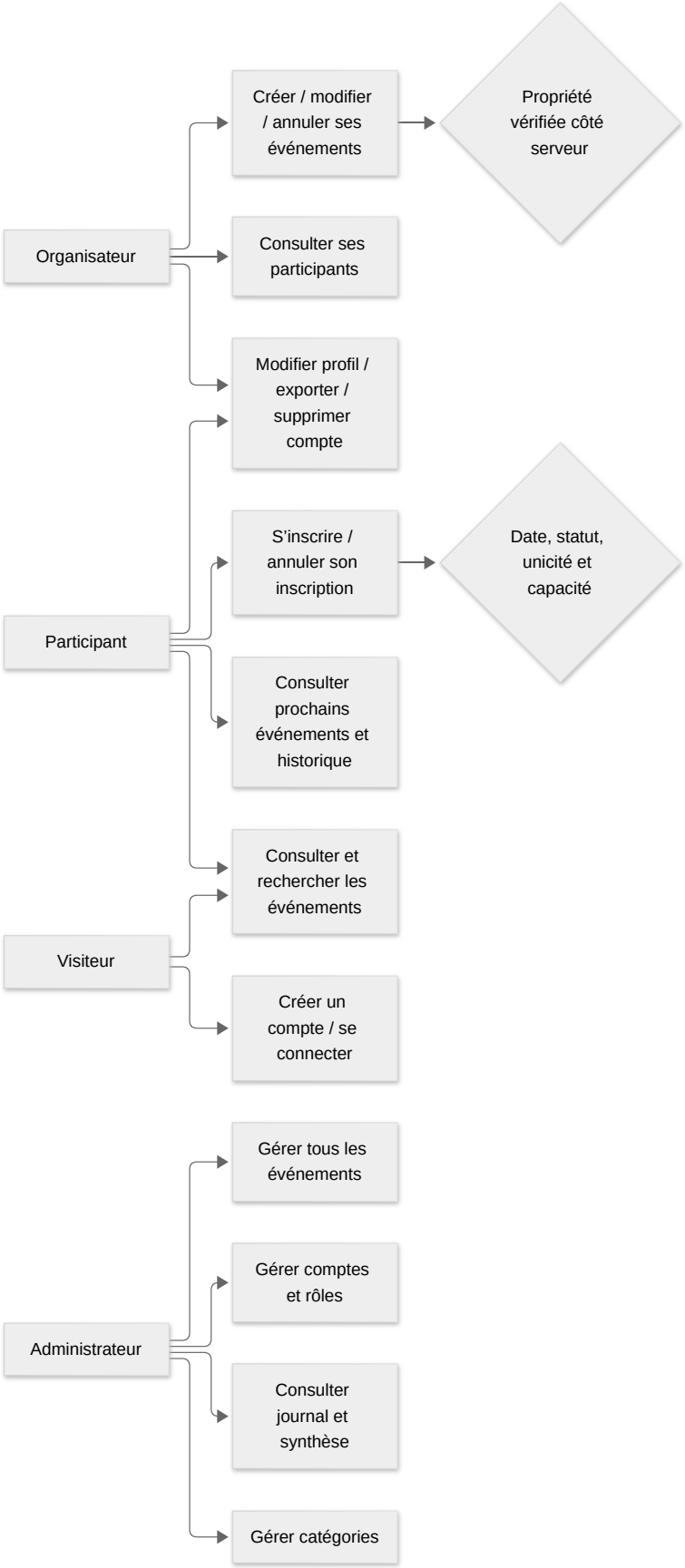
Passage au modèle documentaire

`events.registrations` est un tableau de `{ userId, registeredAt }`. Le couple conceptuel événement/participant est unique grâce au filtre de mise à jour, pas à un index unique multikey (qui ne garantirait pas l'unicité dans un même document). Les coordonnées du participant ne sont pas recopiées dans les événements. L'API publique retourne une vue sans le tableau ni les identifiants des participants.

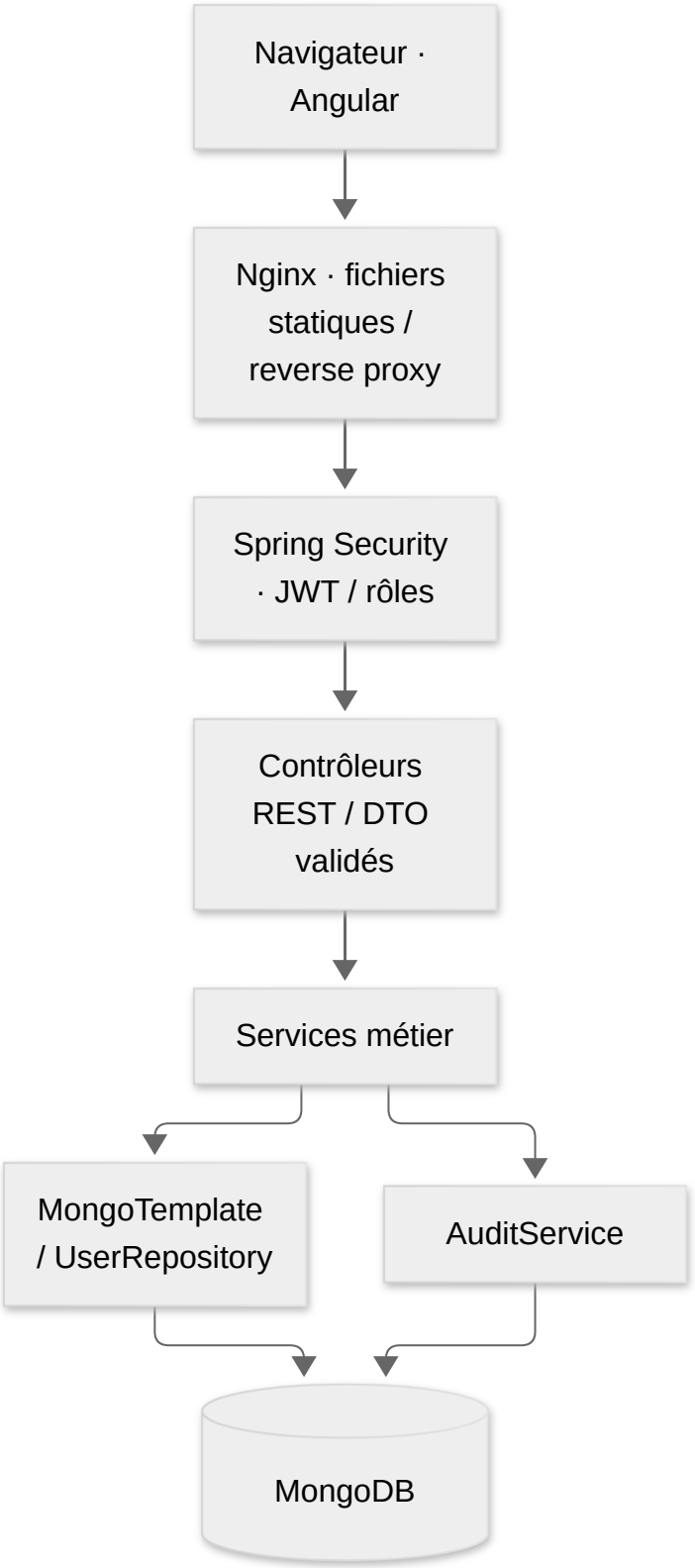
Indexes : `users.email` unique, `categories.normalizedName` unique, date et organisateur des événements, TTL sur `audit.createdAt` (90 jours). Les utilisateurs historiques doivent être normalisés avant création de l'index e-mail ; résoudre les collisions explicitement, sans suppression automatique de comptes. Un index unique simple n'est pas insensible à la casse ; c'est la normalisation à chaque écriture qui l'assure.

La suppression d'une catégorie est remplacée par une désactivation pour conserver les références. L'effacement d'un utilisateur retire ses inscriptions et ses données personnelles ; les événements restent avec un identifiant organisateur non résolu et sans coordonnées.

Cas d’utilisation

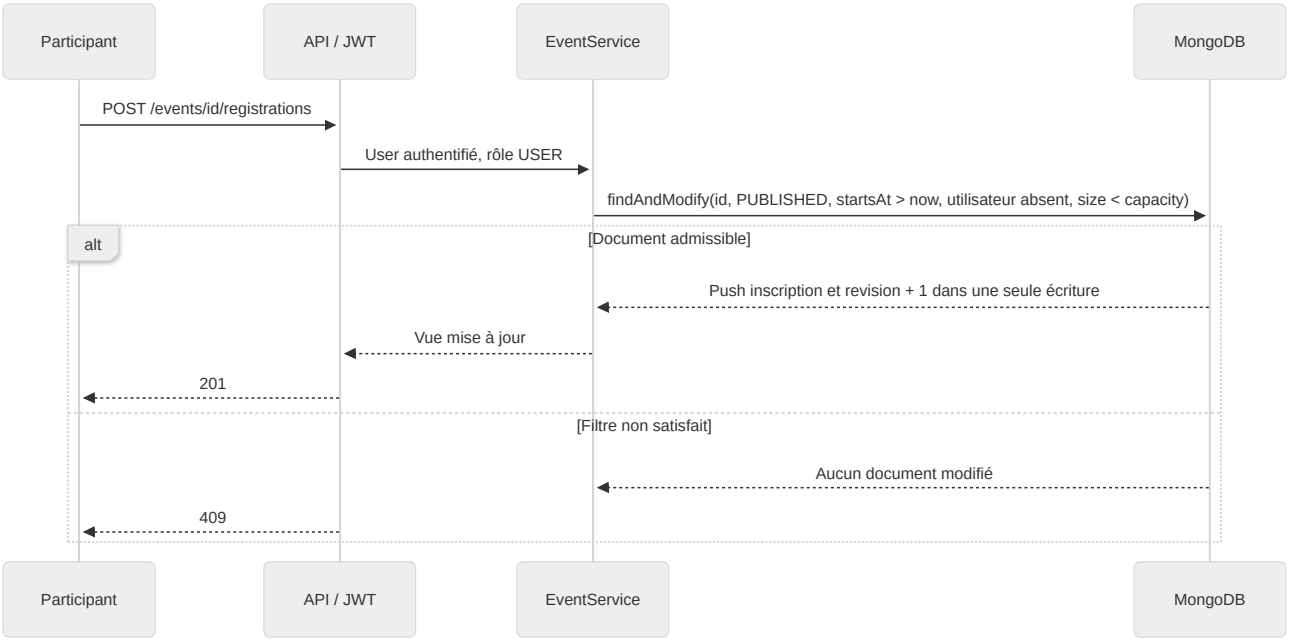


Architecture



Contrôleurs : adaptation HTTP et validation. Services : règles métier et autorisations. Modèles : documents persistants. DTO/vues : exposition contrôlée. Angular ne décide jamais des permissions réelles : les gardes servent uniquement à l’ergonomie.

Inscription atomique



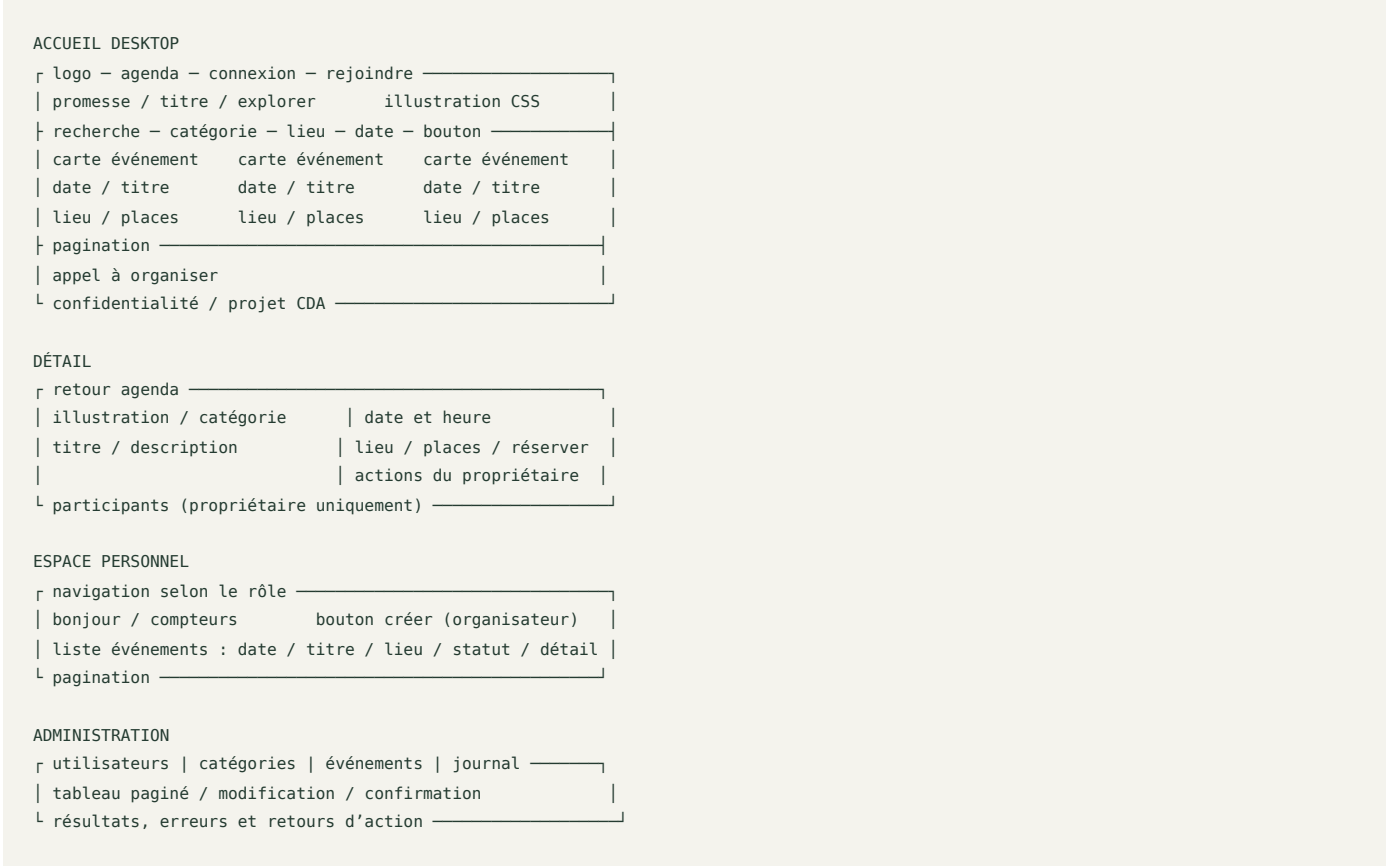
L’annulation, la réservation et l’édition portent sur le même document. L’édition compare la révision ; les réservations incrémentent cette révision. Pas de lecture puis écriture non protégée pour le dernier siège. Aucun verrou Java en mémoire : la garantie fonctionne sur plusieurs instances d’API. Les tests lancent des requêtes HTTP concurrentes réelles.

Les opérations d’effacement d’un compte touchent plusieurs documents et sont des opérations de maintenance réessayables, pas une transaction globale. Pour une exploitation à fort trafic, prévoir une désactivation préalable et une file de purge contrôlée pour traiter les requêtes déjà en vol, ainsi qu’un rapprochement des références orphelines.

Maquettes et expérience utilisateur

La direction visuelle est une interface française « Rassemble » : fond crème, vert profond, accents terracotta, titres sérif et composants simples. Les illustrations sont réalisées en CSS ; aucun service externe, police distante ou image tierce n'est nécessaire.

Les diagrammes suivants représentent la structure des écrans. Les captures réelles sont produites dans `artifacts/screenshots` par le parcours Playwright ; elles ne constituent pas une recherche utilisateur historique inventée.



Accessibilité et adaptation

- Labels associés aux champs, navigation clavier et focus visible.
- Lien d'évitement, navigation sémantique, boutons de taille tactile.
- Retours d'erreur `role=alert`, confirmations `role=status`.
- Les illustrations décoratives sont masquées aux technologies d'assistance.
- Grilles ramenées à une colonne sur mobile ; tableaux défilants.
- Les places, erreurs et statuts sont exprimés par du texte, pas seulement une couleur.
- Confirmations avant suppression, désinscription et annulation.

Tests automatiques : parcours desktop et contrôle du débordement à 390 px. Une recette clavier complète, des mesures de contraste exhaustives et un audit RGAA restent des vérifications humaines à effectuer ; aucune certification d'accessibilité n'est revendiquée.

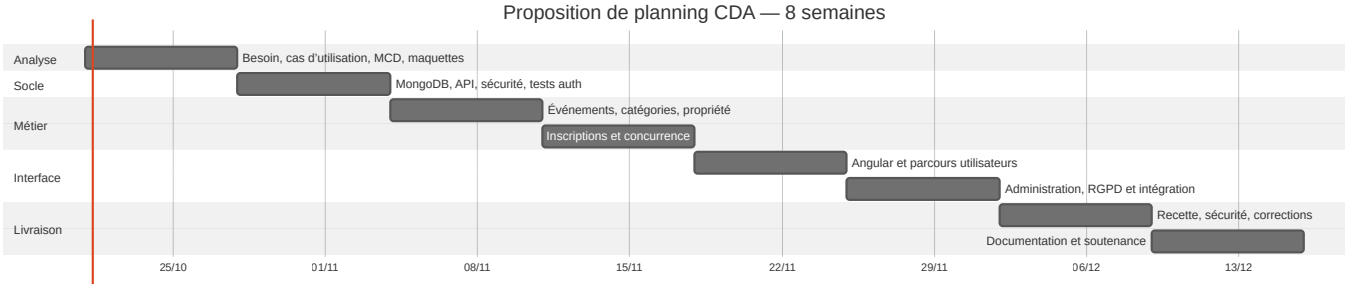
Gestion de projet

Méthode

Backlog priorisé par valeur métier et risques ; itérations courtes avec une démonstration fonctionnelle et des critères de validation. Ce document formalise une organisation possible ; il ne prétend pas qu'une équipe Scrum, un client ou des réunions historiques ont existé. Utiliser des branches de travail et des PR pour les évolutions futures ; aucun push ou ticket externe n'est effectué automatiquement.

Gantt prévisionnel (8 semaines)

Les dates sont un scénario proposé à partir du 21 octobre 2026, à adapter au calendrier réel de formation. Ce Gantt n'est pas une feuille de temps de travail réalisé.



Backlog livré à recetter

Ticket	Priorité	Critère d'acceptation	Preuve technique
AUTH-01 Comptes/JWT	P0	inscription, login et révocation	tests auth HTTP
AUTH-02 Rôles	P0	jamais de rôle choisi à l'inscription	tests privilèges
EVT-01 Agenda	P0	filtres et pagination bornés	tests recherche + navigateur
EVT-02 Gestion	P0	propriétaire ou admin seulement	tests propriété
REG-01 Places	P0	aucune surcapacité simultanée	12 requêtes / 3 places
REG-02 Unicité	P0	un utilisateur inscrit une fois	6 requêtes simultanées
ADM-01 Administration	P0	gestion comptes, catégories, événements	API + écran admin
UI-01 Interface française	P0	parcours complet, responsive	Playwright / captures
DATA-01 Données personnelles	P0	export, correction, suppression	test effacement
OPS-01 Livraison	P0	build et lancement documentés	Docker / CI
DOC-01 Dossier CDA	P0	schémas, tests, sécurité, bilan	dossier docs

Statut : implémenté dans la livraison ; l’acceptation par l’utilisateur et l’enseignant reste distincte. Les résultats de commandes sont recensés dans EVIDENCE.md.

Risques

Risque	Réponse
Périmètre trop vaste	MVP limité, extensions hors engagement
Surcapacité ou doublons	Mise à jour MongoDB atomique, tests concurrents
Escalade de privilèges	rôle initial fixe, contrôles serveur, réauthentification après changement
Écart SQL / critères de certification	faire valider MongoDB seul par l’enseignant
Fuite de données dans le dossier	comptes fictifs, pas de secrets ni adresses réelles dans les preuves
Perte de données	volume, sauvegardes, test de restauration
Documentation prise pour une expérience réelle	éléments personnels et validations marqués à compléter

Contrats API

Base : `/api`. JSON ; dates ISO 8601 avec fuseau (`startsAt` en UTC). Jeton dans `Authorization: Bearer …`. Aucun cookie de session. Les réponses utilisateurs n'exposent ni mot de passe ni version interne du jeton.

Méthode / route	Accès	Objet
POST /auth/register	public	firstName, lastName, email, password ; 201
POST /auth/login	public	email/password ; token et user
POST /auth/logout	authentifié	révoque les jetons du compte ; 204
GET /me	authentifié	profil courant
PUT /me	authentifié	firstName et lastName ; 204
PUT /me/password	authentifié	currentPassword/newPassword ; révocation ; 204
GET /me/export	authentifié	profil et vues des événements personnels
DELETE /me	authentifié hors admin	effacement ; 204, 409 si événements à venir
GET /events	public selon scope	liste paginée
GET /events/{id}	public	fiche sans données des participants
POST /events	organisateur/admin	EventInput ; 201
PUT /events/{id}	propriétaire/admin	EventInput et revision ; 200
DELETE /events/{id}	propriétaire/admin	annulation logique ; 204
POST /events/{id}/registrations	participant	réservation atomique ; 201
DELETE /events/{id}/registrations/me	inscrit	désinscription avant le début ; 204
GET /events/{id}/participants	propriétaire/admin	noms et dates d'inscription
GET /categories	public	catégories actives et inactives
POST /admin/categories	admin	name/active ; 201
PUT /admin/categories/{id}	admin	name/active
GET /admin/users	admin	comptes paginés
PUT /admin/users/{id}	admin	role/disabled ; révoque les jetons
GET /admin/audit	admin	actions paginées
GET /dashboard	authentifié	compteurs selon rôle

Méthode / route	Accès	Objet
GET /health	public	application prête et ping MongoDB valide

`GET /events` : `q` (titre littéral), `category` (id), `location` (littéral), `from`, `to` (ISO Instant), `page` (0 par défaut), `size` (12, maximum 100). `scope=public` masque les annulés ; `mine` exige organisateur/admin, `registrations` exige connexion, `admin` exige administrateur. La réponse est `{items,total,page,size}`. Les tableaux admin utilisent la même convention, avec une taille par défaut de 20.

Exemple EventInput :

```
{"title":"Atelier créatif","description":"Une rencontre ouverte à tous.","startsAt":"2027-06-
01T16:00:00Z","location":"Nantes","categoryId":"id-existant","capacity":20,"revision":0}
```

Les révisions changent après modification, inscription, désinscription ou annulation. Un 409 à l'édition demande de recharger la fiche, pas de répéter aveuglément la requête.

Erreurs : 400 validation, 401 authentification/identifiants, 403 droits, 404 ressource, 409 conflit, 429 limitation au niveau du reverse proxy. Les erreurs métier utilisent `application/problem+json` avec `status` et `detail`. Les validations ajoutent `errors`. Les réponses inattendues restent gérées par Spring sans stacktrace ni détails internes.

Sécurité, OWASP 2025 et RGPD

Référence demandée : https://owasp.org/Top10/2025/. Le site a répondu HTTP 403 dans cet environnement ; la nomenclature a été vérifiée dans le dépôt officiel OWASP : https://github.com/OWASP/Top10/blob/master/2025/docs/en/index.md (consulté le 21 octobre 2026). Ce tableau est une analyse de risques et de contrôles, pas une certification de conformité.

OWASP 2025	Contrôles présents	Compléments d'exploitation
A01 Broken Access Control	rôles serveur, propriété, vues sans participants, tests 401/403	recette des habilitations réelles
A02 Security Misconfiguration	pas de stacktrace, réseau Docker interne, utilisateur non-root, en-têtes	TLS, secrets, configuration de l'hôte
A03 Software Supply Chain Failures	lock npm, Gradle wrapper, CI, dépendances versionnées	audit récurrent, signatures et pins d'images/actions
A04 Cryptographic Failures	BCrypt, JWT signé et expirant, secrets hors Git	TLS obligatoire, chiffrement sauvegardes/disques
A05 Injection	DTO typés, recherche regex échappée, Angular interpolation	test offensif, limites de charge
A06 Insecure Design	règles capacité/duplication atomiques, droits documentés	revue indépendante, modèle de menaces élargi
A07 Authentication Failures	mots de passe 12 caractères, erreurs génériques, JWT révoqués, comptes désactivables	MFA et récupération de compte hors MVP
A08 Software or Data Integrity Failures	index uniques, contrôle de révision, pas de rôle venant de l'inscription	provenance des artefacts de livraison
A09 Security Logging and Alerting Failures	audit minimal, TTL 90 jours, absence de secrets	centralisation, alertes, procédure d'incident
A10 Mishandling of Exceptional Conditions	ProblemDetail, conflits explicites, tests de refus, audit non bloquant	résilience DB, tests de panne et supervision

Choix d'authentification

Le JWT est stocké seulement en mémoire du navigateur et expire après une heure. Un rechargement demande une reconnexion. La signature contient l'identifiant du compte et une version ; chaque requête vérifie le compte et sa version en base. Déconnexion, changement de mot de passe et modification des droits invalident les anciens jetons. Les rôles sont relus côté serveur, pas acceptés depuis le navigateur.

CSRF est désactivé parce que l'API utilise uniquement un en-tête Bearer explicitement ajouté ; aucune authentification par cookie n'est utilisée. L'interface et l'API sont de même origine. Les contrôles CORS permissifs ne sont pas activés.

Le reverse proxy Docker limite les tentatives login/inscription (10 requêtes/minute et burst 10 par adresse IP). L'API doit rester sur le réseau interne ; le serveur de développement n'offre pas cette protection. En cas de proxy supplémentaire, configurer l'adresse client de confiance sans accepter aveuglément un X-Forwarded-For utilisateur.

Le contrôle de longueur BCrypt est fait en octets UTF-8 (72 maximum), en plus de la validation en caractères. Le jeton et le hash du mot de passe ne figurent jamais dans les vues ou journaux. L'échec d'audit est signalé dans les logs techniques sans annuler une opération métier déjà réalisée.

RGPD : minimisation et droits

Données	Finalité	Exposition / conservation
prénom, nom, email, hash	compte et service demandé	compte jusqu’à suppression ; accès limité
événement, inscription	organisation et réservation	compte et organisateur concerné (noms seulement)
acteur/action/ressource/date	diagnostic et sécurité	administrateurs ; TTL 90 jours
JWT en mémoire	authentification	fin de page/session ou une heure, révocable

Fonctions : rectification du nom, export JSON, effacement du compte et de ses inscriptions, effacement des traces d’audit liées. Pas de marketing ni traceur. Les acteurs sont identifiés par identifiants techniques dans le journal (pseudonymisation, pas anonymisation absolue). Les sauvegardes exigent une politique d’expiration et de réapplication des suppressions après restauration.

Avant un usage réel, l’exploitant doit renseigner : responsable et contact, base légale adaptée (exécution du service pour le compte, intérêt légitime documenté pour les traces ou autre fondement approprié), sous-traitants/hébergement, transferts, registre, durée métier des événements et demandes de droits. Une page de confidentialité et des endpoints ne suffisent pas à établir la conformité RGPD.

Points à durcir avant ouverture publique

TLS, compte MongoDB applicatif à privilèges minimaux (Compose utilise un compte root de démonstration), gestionnaire de secrets, surveillance et rotation, limitation API globale, MFA admin, scans de dépendances avec traitement des alertes, audit d’accessibilité, tests de charge, procédure de purge résiliente et contacts RGPD. Les images Docker sont versionnées par majeure ; les figer par digest dans une livraison d’exploitation revue.

Stratégie de tests et qualité

Les tests d'intégration démarrent une vraie API sur un port aléatoire et communiquent avec un vrai MongoDB local. Ils ne remplacent pas la persistance par des mocks. Une base `platform_test_<uuid>` est créée puis supprimée ; le test de contexte utilise une base sans données métier. Exécuter MongoDB avant Gradle.

Scénario	Test automatisé
disponibilité MongoDB et application prête	healthChecksApplicationReadinessAndMongoDB
JWT expiré et signature altérée	expiredAndTamperedTokensAreRejected
inscription, hash, normalisation, unicité, rôle initial	authenticationValidationAndEmailUniqueness
endpoints publics/protégés et privilèges	anonymousAndInsufficientRolesAreRejected
déconnexion et changement mot de passe	logoutAndPasswordChangeRevokeTokens
propriétaire, tiers, date passée et révision obsolète	eventOwnershipValidationAndOptimisticConcurrency
complet, double inscription, désinscription, annulé, historique	registrationCapacityDuplicatesCancellationAndHistory
RM02 / RM05 : 12 requêtes pour 3 places	simultaneousRegistrationsNeverOversell
RM03 : 6 réservations du même utilisateur	simultaneousDuplicateRegistrationIsAtomic
réduction capacité et événement passé	capacityCannotBeReducedBelowRegistrationsAndPastEventsRejectRegistration
rôle et compte désactivé, protection admin	adminRoleChangesAndDisabledAccountsInvalidateExistingTokens
recherche littérale, catégories inactives, pagination, audit	categoriesSearchPaginationAndAudit
profil, export sans secrets, effacement et références	profileExportAndErasure

Tests Playwright : agenda/filtres/mobile ; création de compte participant, réservation, profil/export/effacement ; publication/modification/annulation organisateur ; administration catégories/comptes/journal. Les tests utilisent une base jetable distincte et des données fictives, puis la suppriment. Les captures produites montrent ces données seulement.

Commandes et résultats

```
cd backend && bash gradlew test bootJar --no-daemon
cd ../frontend && npm ci && npm run build
# Installer Chromium Playwright une fois, puis revenir à la racine :
npx playwright install chromium
cd .. && bash scripts/e2e.sh
```

Si un Chromium système existe, `CHROMIUM_PATH=/usr/bin/chromium bash scripts/e2e.sh` est une alternative ; aucune vérification TLS n'est désactivée pour télécharger un navigateur.

Rapports : `backend/build/reports/tests/test`, `frontend/playwright-report`, traces des échecs dans `frontend/test-results`, captures `artifacts/screenshots`. Ces sorties sont ignorées par Git ; les exporter et anonymiser avant de les joindre à une soutenance. La CI conserve les rapports en artefacts, mais aucune exécution GitHub distante n'est revendiquée tant qu'elle n'a pas eu lieu.

Recette humaine à compléter

- Naviguer au clavier, vérifier lecteur d'écran, contrastes et retours d'erreur.
- Confirmer les libellés et les décisions métier avec l'enseignant.
- Tester fuseau horaire, deux navigateurs et un mobile physique.
- Simuler perte réseau, panne MongoDB, restauration et rotation de secrets.
- Mesurer le temps de réponse sous une charge représentative du futur hébergement.

Pas de pourcentage de couverture inventé. La réussite de ces tests prouve les scénarios exécutés, pas l'absence de toute vulnérabilité ni l'aptitude à une charge inconnue.

Dans GitHub Actions, MongoDB démarre via `scripts/ci-mongo.sh` après le checkout, avec un ping explicite avant les tests. En cas de panne, consulter l'étape « Start MongoDB and wait for readiness » et `mongo-ci.log` dans les artefacts. Les tests ne sont ni ignorés ni remplacés en cas de base indisponible.

Preuves et traçabilité

Ce fichier recense uniquement les vérifications effectivement exécutées dans l'environnement courant. Les résultats finaux figurent dans le tableau ajouté après validation.

Sources des preuves : code et diff Git, rapports JUnit, rapport Playwright, captures de parcours avec données fictives, builds, configuration de déploiement. Les artefacts volumineux sont ignorés par Git ; les joindre à la remise si nécessaire. Aucun historique de commits, ticket externe, réunion ou retour client n'a été fabriqué.

Ne jamais joindre `.env`, les variables de processus, les mots de passe, les jetons ou les logs complets non relus. Les captures test utilisent exclusivement des comptes fictifs.

Collecte proposée

Preuve	Emplacement	Interprétation
Résultats JUnit	backend/build/test-results/test/*.xml	tests terminés et nombre exact
Rapport lisible	backend/build/reports/tests/test	détail des scénarios API
Rapport navigateur	frontend/playwright-report	parcours frontend/API réels
Captures	artifacts/screenshots	interface avec données de démonstration
Tests et règles	docs/TESTS.md	correspondance exigences/vérifications
Configuration CI	.github/workflows/ci.yml	pipeline livré ; exécution distante non attestée
Source présentée	git status / git diff / git rev-parse HEAD	identité réelle du code, pas d'historique inventé

Validation finale — 21 octobre 2026

Vérification exécutée	Résultat observé
JUnit / API réelle / MongoDB	**14 tests passés, 0 échec, 0 ignoré**
Playwright / Chromium	**4 parcours passés, 0 échec, 0 ignoré**
Build backend et JAR	réussi
Build Angular de production	réussi ; bundle initial environ 404 ko
Formatage Java / Prettier / diff Git	vérifications passées
Audit npm, dépendances de production	0 vulnérabilité connue au moment du contrôle

Vérification exécutée	Résultat observé
Images Docker backend et frontend	construites avec TLS vérifié et secrets BuildKit temporaires
Application Docker / MongoDB authentifié	parcours connexion, réservation, tableau de bord et déconnexion validé
Démarrage natif scripts/dev.sh	proxy Angular, readiness et parcours compte validés ; processus de test arrêtés
Jeu fictif scripts/seed-demo.mjs	exécuté puis réexécuté sans doubler les six événements
Diagrammes Mermaid	5 diagrammes interprétés et exportés en SVG
CI GitHub distante	échec de démarrage MongoDB signalé par l'utilisateur ; correction du démarrage testée localement, nouvelle exécution distante à vérifier
Publication cloud / mise en production	non effectuée

Le site OWASP a refusé l'accès direct (403) ; la liste 2025 a été vérifiée à partir du dépôt GitHub officiel OWASP. Les téléchargements Playwright étant interdits sur ce réseau, les tests ont utilisé Chromium système, sans désactivation de vérification TLS. Un défaut de permissions de configuration Nginx a été corrigé puis le conteneur retesté.

Résultats détaillés : verification.json, manifeste des sources et rapports JUnit anonymisés dans evidence/junit. Le manifeste identifie les sources vérifiées, sans prétendre que les changements ont été commités ou poussés.

Captures réellement produites (données fictives)

- Agenda desktop — six événements.
- Agenda mobile.
- Tableau de bord participant.
- Détail d'événement.
- Espace organisateur.
- Administration.
- Journal d'activité.

Ces preuves témoignent de la livraison technique dans cet environnement ; elles ne remplacent pas l'acceptation pédagogique, l'analyse personnelle, l'audit de production ou les tests humains d'accessibilité. Les configurations privées restent hors du dépôt.

Aperçus intégrés

Aller au contenu
✳ rassemble.
L'agendaConnexionNous rejoindre →

SE RETROUVER. DÉCOUVRIR. PARTAGER.

Les belles rencontres commencent *ici.*

Un atelier, un concert, une nouvelle passion.
Trouvez votre prochain rendez-vous.

Explorer les événements →
À VIVRE ENSEMBLE**Faites place à l'inattendu.**
✳
De nouvelles idées, de nouveaux liens.
ON Y VA ? →

L'AGENDA DES CURIOSITÉS

Votre prochaine découverte

6 événement(s)
Rechercher Un titre, une envie… Catégorie Toutes les catégories Lieu Ville ou lieu À partir du jj/mm/aaaa Rechercher
Ateliers✳24oct.

samedi 24 octobre · 17:00

Les mains dans la terre

→ Maison des arts · Nantes

18 places disponibles→
Culture⌄27oct.

mardi 27 octobre · 17:00

La ville à travers l'objectif

→ Île de Nantes

16 places disponibles→
Sport✦30oct.

vendredi 30 octobre · 17:00

Un dimanche au grand air

→ Parc de Procé · Nantes

30 places disponibles→
Rencontres✳02nov.

lundi 2 novembre · 17:00

Des idées autour d'un café

→ Café des voisins · Nantes

12 places disponibles→
Culture⌄06nov.

vendredi 6 novembre · 17:00

Une soirée, de nouveaux sons

→ Studio collectif · Nantes

25 places disponibles→
Ateliers✦10nov.

mardi 10 novembre · 17:00

Le goût du fait maison

→ Cuisine partagée · Nantes

10 places disponibles→
← Précédent Page 1 Suivant →
✳

LES MEILLEURES IDÉES SE PARTAGENT

Et si le prochain événement, c'était le vôtre ?

Créez votre compte, puis contactez l'administrateur pour devenir organisateur.

Rejoindre Rassemble →
✳ rassemble.

Des événements. Des gens. Des liens.

Confidentialité & données

Projet CDA · Une plateforme pensée pour se retrouver.

Aller au contenu
✳ rassemble.
L'agendaMon espaceMon profilDéconnexion
Votre place est réservée. À bientôt !×

VOTRE ESPACE PERSONNEL

Bonjour, Alex.

Vos prochains moments à partager.

1Inscriptions
1À venir

Mes inscriptions & mon historique

Explorer l'agenda →
24oct.

Les mains dans la terre

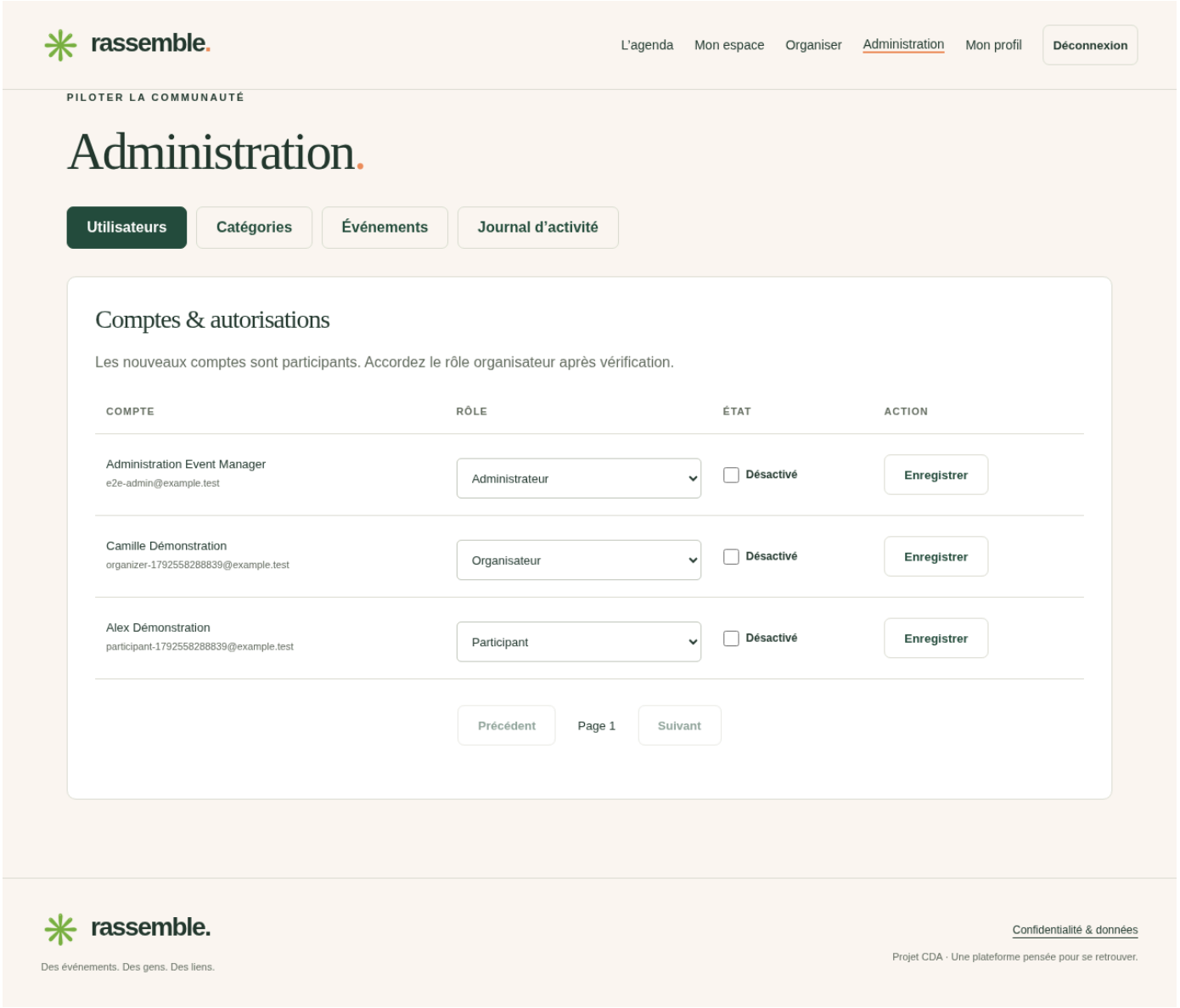
Maison des arts · Nantes · 17:00

Inscrit→
Précédent Page 1 Suivant
✳ rassemble.

Des événements. Des gens. Des liens.

Confidentialité & données

Projet CDA · Une plateforme pensée pour se retrouver.



Suivi du démarrage CI

L'utilisateur a signalé un échec d'initialisation du service MongoDB avant exécution des tests GitHub. L'image et l'ancien health check ont démarré correctement en local ; la cause distante exacte n'est donc pas établie sans les logs du conteneur. Le workflow utilise désormais `scripts/ci-mongo.sh` après checkout : attente de readiness explicite (120 s maximum), erreur si le processus quitte, conservation des logs dans les artefacts des deux jobs et nettoyage en fin de job. Le script et une connexion MongoDB depuis l'hôte ont été validés sur un conteneur isolé local. Ce changement ne transforme pas une panne de base en test réussi ; une nouvelle exécution GitHub reste nécessaire pour confirmer le résultat distant.

Cause confirmée de la panne CI

Les logs fournis par l'utilisateur identifient la cause : l'image MongoDB 8.0 (digest `4a4ab5d4834207389e0d7270fac988e21ffedccecba77991a552f755df229907`) refuse de démarrer sur un noyau Linux 6.19 ou ultérieur (MongoDB SERVER-121912). Le label `ubuntu-latest` migrait vers Ubuntu 26. Les deux jobs sont désormais fixés sur `ubuntu-24.04` ; le manifeste officiel runner-images consulté indique un noyau 6.17.0-1022-azure pour cette image. Les actions checkout, setup-java, setup-node, upload-artifact et setup-gradle utilisent des versions avec runtime Node 24 vérifié dans leurs manifestes officiels. Le résultat de la nouvelle exécution GitHub reste à confirmer ; aucune exécution distante réussie n'est affirmée depuis cet environnement.

Révision après échec du job frontend

Une capture ultérieure montre le backend réussi et le job frontend refusé par MongoDB pour le même noyau >= 6.19. L'horodatage est compatible avec le run du commit 808c2fa ; le passage à Ubuntu 24.04 n'est donc pas considéré comme une correction distante validée. Les

deux jobs sont désormais fixés sur Ubuntu 22.04 ; le manifeste officiel runner-images consulté indique 6.8.0-1064-azure. Le helper affiche `uname -sr` pour identifier le noyau effectif. L'application et les tests ne sont pas modifiés ni désactivés. Le nouveau résultat distant reste à confirmer.

Livraison, maintenance et exploitation

Configuration

Java 21 complet, Node 24 pour le build, MongoDB 8. Le wrapper Gradle versionne l'outil de build ; npm utilise le lockfile via `npm ci`. MongoDB seul ; aucun serveur SQL. Le backend reçoit `MONGODB_URI`, `JWT_SECRET`, et éventuellement `ADMIN_EMAIL`/ `ADMIN_PASSWORD` pour amorcer le premier compte. Le secret JWT doit être aléatoire, privé et d'au moins 32 octets. Aucun secret livré en dur.

Le compte d'amorçage n'est créé que s'il n'existe pas. Ne pas confondre un changement de variable avec une réinitialisation de mot de passe. Après amorçage, retirer le mot de passe admin des variables quand le mode d'exploitation le permet et utiliser le profil pour le changer. Une base existante peut exiger une migration des e-mails : normaliser et résoudre les doublons avant l'index unique.

Docker Compose

Voir le README. `docker compose up --build -d`, puis vérifier l'API via le proxy et effectuer une vraie connexion. Le ping MongoDB et un processus HTTP ne prouvent pas le fonctionnement métier : créer une catégorie, promouvoir un organisateur, publier et réserver une place. Ne pas ouvrir les ports MongoDB/API au réseau public. Le bind web par défaut est local ; ajouter un reverse proxy TLS administré pour une publication réelle.

`docker compose down` arrête les conteneurs sans supprimer le volume. **Ne pas ajouter `-v` sur une base à conserver.** Une modification des identifiants MongoDB dans `.env` ne réinitialise pas les utilisateurs d'un volume déjà initialisé : effectuer une rotation avec MongoDB, puis mettre à jour la configuration de l'API.

Sauvegarde / restauration

Depuis un terminal administré, sans journalisation shell des secrets :

```
mkdir -p backups
# Utilise les variables déjà injectées dans le conteneur, sans les afficher.
docker compose exec -T mongo sh -c 'mongodump --username "$MONGO_INITDB_ROOT_USERNAME" --password "$MONGO_INITDB_ROOT_PASSWORD" --
authenticationDatabase admin --db event_manager --archive --gzip' > backups/event-manager.archive.gz
# Restaurer uniquement dans une instance de recette isolée après validation de la cible.
# mongorestore --archive=... --gzip avec les identifiants de cette instance.
```

Chiffrer les sauvegardes, restreindre l'accès, définir leur durée et tester régulièrement une restauration. Le mot de passe est passé à l'outil interne ; limiter aussi les droits d'accès système au conteneur. La restauration réelle n'est pas attestée sans preuve d'exécution. Réappliquer les demandes d'effacement après une restauration ancienne.

Maintenance

1. Reproduire un défaut avec un ticket, impact et étapes précis ; exclure les données réelles.
2. Ajouter un test utile qui échoue, corriger sur une branche, relire la PR.
3. Exécuter tests API et navigateur, build frontend/backend, revue des dépendances.
4. Sauvegarder, déployer la nouvelle version, vérifier un parcours métier et les logs.
5. Revenir à l'image précédente si échec ; ne pas restaurer une base sans analyser les modifications de schéma et les données écrites depuis la livraison.

Surveiller disponibilité, erreurs HTTP, espace disque MongoDB, échecs de login et de journalisation. Le TTL MongoDB est asynchrone ; 90 jours n'est pas une échéance à la seconde près. Les erreurs applicatives n'exposent pas les stacktraces au navigateur.

Limitations opérationnelles

Compose privilégie une démonstration reproductible, pas une infrastructure hautement disponible. Le compte MongoDB root doit être remplacé par un utilisateur applicatif limité avant production. Pas de réplication, monitoring central, sauvegarde automatique, TLS managé ni rotation automatique. Les opérations de purge multi-documents sont à renforcer avant forte concurrence. Le guide décrit les compléments requis sans prétendre les avoir mis en exploitation.

Construction derrière un proxy de confiance

Les Dockerfiles acceptent des secrets BuildKit optionnels : `gradle_properties` pour les propriétés proxy Gradle, `java_truststore` pour le magasin de certificats Java, et `proxy_ca` pour la CA Node. Ils ne sont pas incorporés aux images. Dans un réseau cloud où le nom du proxy n'est pas résolu depuis Docker, fournir aussi un `extra_hosts` de build avec son adresse actuelle. Ne pas copier de jeton dans un ARG, ne pas désactiver TLS et ne pas désactiver les checksums. Le wrapper vérifie le SHA-256 officiel de sa distribution Gradle, enregistré dans gradle-wrapper.properties.

Compatibilité du noyau avec MongoDB

L'image MongoDB 8.0 utilisée dans cette livraison refuse les noyaux Linux 6.19 ou ultérieurs (voir https://jira.mongodb.org/browse/SERVER-121912). Les conteneurs partagent le noyau hôte : Docker ne contourne pas cette incompatibilité. Utiliser un hôte compatible ou une version MongoDB officiellement corrigée et revalidée. La CI est fixée sur Ubuntu 22.04 (noyau 6.8 dans son manifeste officiel). Une première fixation sur Ubuntu 24.04 n'a pas suffi sur le runner du job frontend ; le noyau effectif est désormais affiché dans les logs de démarrage.

Dossier de remplacement de stage et préparation de soutenance

Présentation à personnaliser

Faits connus : projet individuel de Nicolas Patigny, formation CDA ENI, plateforme de gestion d'événements, remplacement de stage. Le besoin et le périmètre proviennent du cahier des charges et des consignes de l'enseignant.

À compléter honnêtement : structure réelle si applicable, statut du projet pédagogique, interlocuteurs, dates effectives, contraintes matérielles, modalités d'accompagnement, validation du sujet et du choix MongoDB seul. Ne pas inventer un client, une entreprise, un tuteur ou des observations de terrain.

Missions et trois blocs CDA

Les intitulés ci-dessous suivent l'organisation courante du titre CDA ; vérifier le code RNCP et la version du référentiel de la promotion avant remise.

Bloc	Missions / livrables	Preuves à présenter
Développer une application sécurisée	environnement, Angular, formulaires, composants métier, Git et tests	code, parcours filmé, rapports et captures
Concevoir et développer une application sécurisée organisée en couches	analyse, MCD, architecture REST/services/persistance, contrôle des données	DESIGN, API, mises à jour atomiques et tests concurrents
Préparer le déploiement d'une application sécurisée	builds, CI, conteneurs, configuration, recette et maintenance	Dockerfiles/Compose, workflow, rapports, procédure de sauvegarde

Le schéma conceptuel ne prouve pas l'exécution d'une base relationnelle. Si le référentiel exige cette compétence, demander à l'enseignant comment la démontrer séparément sans présenter MongoDB comme du SQL. Le dossier doit refléter le travail compris et réalisé par le candidat. L'assistance de génération de code doit être déclarée selon les règles de l'établissement ; relire, expliquer, tester et adapter le code avant soutenance.

Difficultés et solutions techniques observables

Difficulté	Solution dans le projet	Preuve
Deux réservations pour la dernière place	filtre et écriture atomiques dans le document événement	test 12 requêtes / 3 places
Double inscription simultanée	condition utilisateur absent et push indivisibles	test 6 requêtes identiques
JWT valable après déconnexion	version persistée et relue à chaque requête	test révocation
Écrasement d'une modification	révision optimiste sur les éditions	test révision obsolète
Exposition des inscrits	vues publiques séparées et liste réservée au propriétaire	tests autorisations

Difficulté	Solution dans le projet	Preuve
Réseau cloud et navigateur	Chromium système en alternative au téléchargement interdit	configuration Playwright

Ce tableau décrit des problèmes techniques réellement traités, pas des apprentissages personnels attribués automatiquement au candidat.

Journal personnel à remplir

Pour chaque session réelle : date, objectif, travail personnel réalisé, aide reçue, difficulté, hypothèse, vérification, résultat, apprentissage, lien vers preuve anonymisée.

```
Date : [date réelle]
Mission et bloc CDA : [à compléter]
Ce que j’ai réalisé et ce que j’ai compris : [à compléter]
Aide ou outils utilisés : [à compléter]
Problème observé et démarche : [à compléter]
Vérification / preuve : [lien réel]
Ce que je ferais autrement : [analyse personnelle]
```

Analyse personnelle (questions, pas réponses fabriquées)

- Qu’ai-je appris sur MongoDB, les règles métier et les écritures concurrentes ?
- Quelles parties puis-je expliquer et modifier sans assistance ?
- Quel test a changé ma compréhension d’un défaut ?
- Pourquoi ce découpage en couches, et quelles limites ai-je constatées ?
- Qu’est-ce que je simplifierais ou renforcerais avec deux semaines de plus ?
- Comment ai-je organisé mon temps et adapté mon périmètre ?
- Quelle responsabilité reste à l’exploitant en sécurité et protection des données ?

Proposition de soutenance

1. Contexte réel et problème utilisateur (2 min).
2. Périmètre, acteurs, MCD et architecture (3 min).
3. Démonstration participant / organisateur / admin (6 min).
4. Focus inscription atomique et contrôle des droits (3 min).
5. Tests, sécurité, RGPD et livraison (3 min).
6. Bilan personnel, limites et améliorations (3 min).

Ajuster à la durée officielle. Préparer une base fictive et un scénario sans dépendance à un service externe. Capturer les rapports datés, citer le commit réellement présenté, anonymiser les images et ne montrer ni secrets ni adresses de personnes réelles.